



✦ ✦ 作者主页: [嶽某](#) ✦ ✦

✦ ✦ 所属专栏: [MySQL](#) ✦ ✦

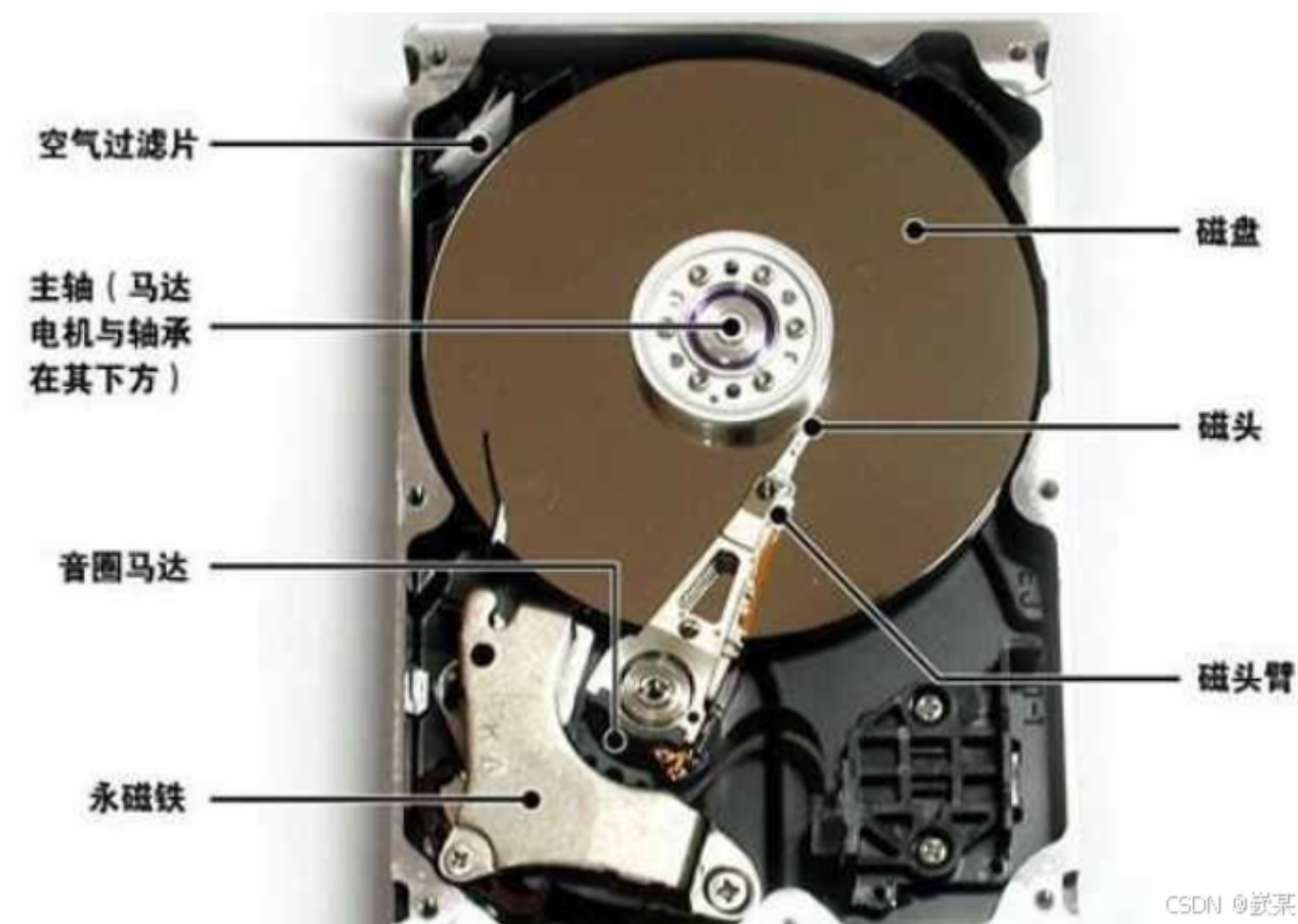
我们知道，数据库里面会有成千上万条数据，我们会对这些数据进行CURD操作，如果我要在海量数据中查找一条特定的数据，不可能让数据库遍历这些数据。那么在数据库底层一定有一种数据结构，以便于我们快速查找，检索信息。

那么这种数据结构是什么？它是怎么和信息匹配到一起的？它和本文要说的索引是什么关系？

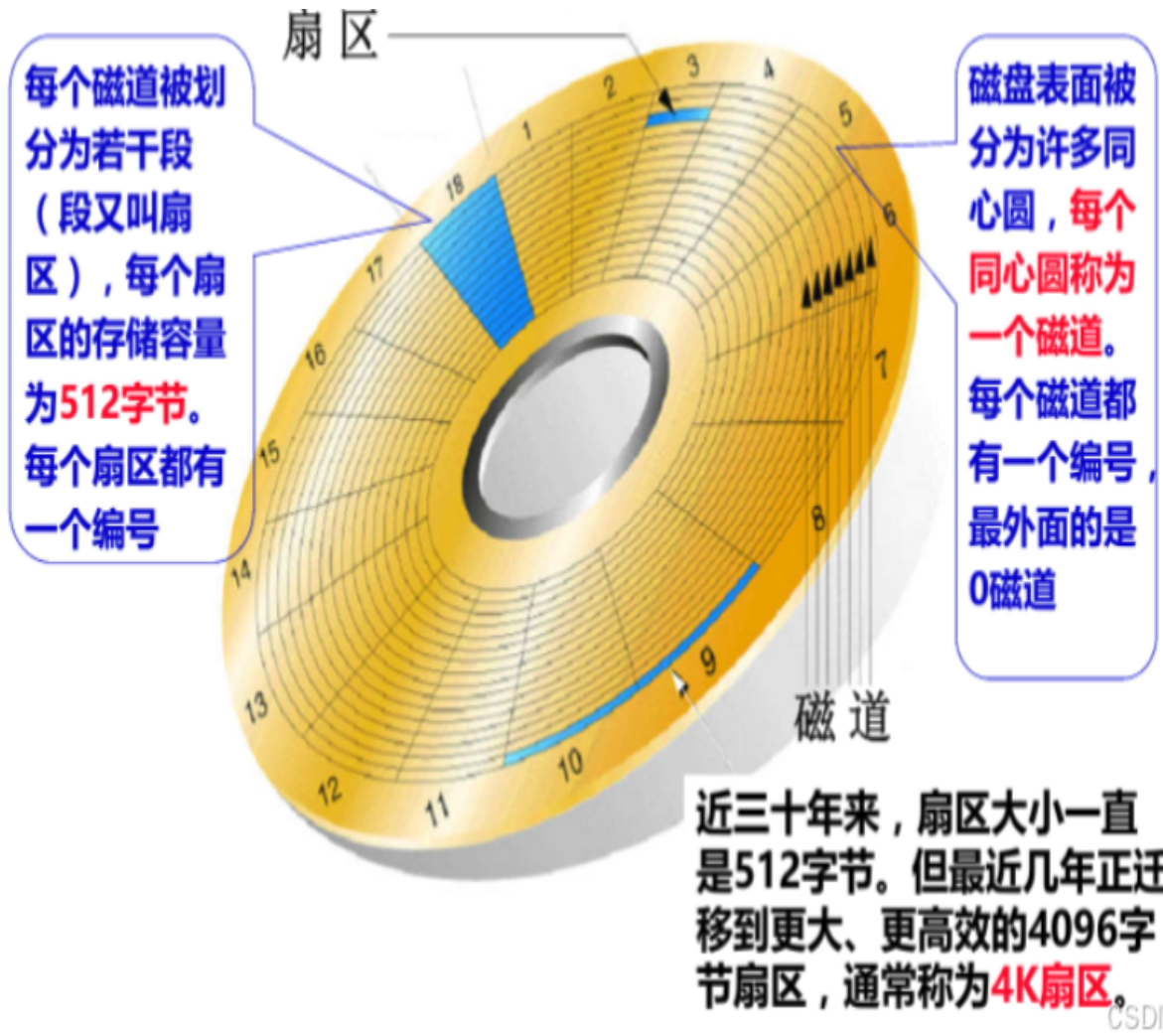
这一切的开始，我们先来认识一下磁盘，也就是数据库存储数据的地方。

认识磁盘

MySQL给用户提供服务，存储的数据都放在磁盘上，磁盘是计算机的一个机械设备，相较于其他电子元件，磁盘的IO效率是较低的。再加上IO本身对操作系统的资源消耗，**如何提高IO效率**就成为了MySQL的一个重要话题。



盘片



CSDN @ 某某

扇区

数据库文件，本质其实就是保存在磁盘的盘片中，也就是上面的一个个小格子中，也就是扇区。数据库很大，也很多，一定需要占用很多个扇区。

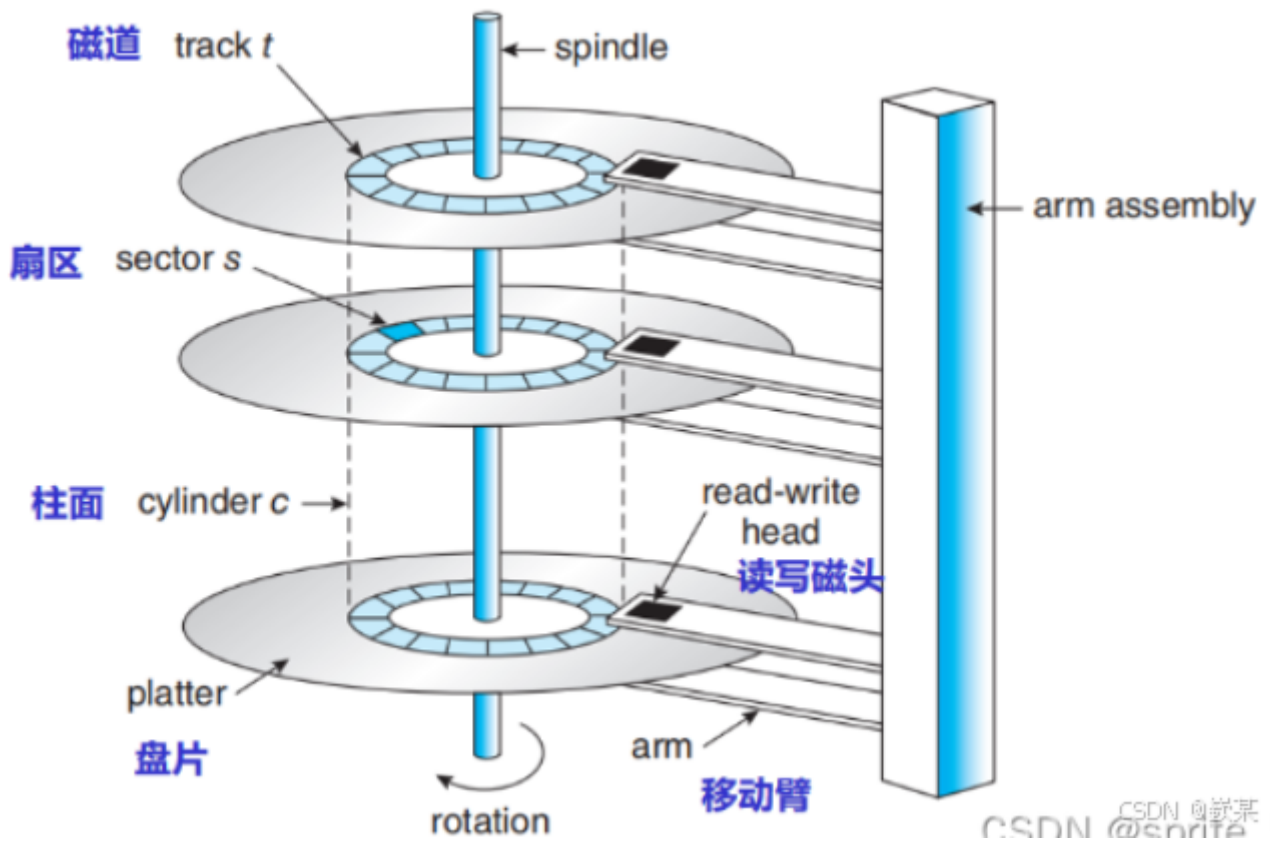
- 从上图可以看出，靠近盘中间的扇区的空间越小，距离圆心越远，扇区越大。
- 那么所有扇区都默认都是512字节吗？目前我们就这样认为，因为一个扇区多大是由比特位密度决定的。
- 但是最新的磁盘技术，已经可以让扇区的大小不同了，但是我们暂时不考虑。

我们在使用Linux，所看到的大部分目录或者文件，其实都是保存在硬盘中的（不考虑内存级文件系统，如proc, sys等）

所以，找到一个文件的全部，本质就是找到其在磁盘上的扇区。

如果我们能定位一个扇区，那么就能找到所有扇区，因为它们的寻找方式都是一样的。

定位扇区



- 柱面（磁道）：多盘磁盘，每盘都是双面，大小完全相等。那么同半径的磁道，整体便构成了一个柱面
- 每个盘面都有一个磁头，盘面和磁头的关系是一对一的
- 所以我们只需要知道，磁头Heads、柱面Cylinder（等价于磁道）、扇区Sector对应的编号。即可在磁盘上定位所要访问的扇区。这种磁盘定位方式叫做CHS。不过实际系统软件使用的并不是CHS，而是LBA，一种线性地址，可以想象成虚拟地址与物理地址。系统将 LBA 地址最后会转化成为 CHS，交给磁盘去进行数据读取。但是这并不重要，知道就好。

那么我们在系统软件上是否就是以 一个扇区（512或4096字节）进行IO交互吗？并不是

- 如果操作系统直接使用硬件提供的数据大小进行交互，那么系统的IO代码和硬件强相关，如果硬件发生变化，系统就必须跟着变。

- 从目前来看，单次512字节还是太小了。这意味这访问同样的数据内容，需要进行更多次的磁盘访问，带来效率降低。
- 文件系统的基本读取单位也不是扇区，而是数据块。

所以**系统读取磁盘，是以块为单位的，基本单位是4KB。**

随机磁盘访问(Random Access)与连续磁盘访问(Sequential Access)

随机访问：本次IO所给出的扇区地址与上次IO给出的扇区地址不连续，这样的话磁头在两次IO操作之间需要做比较大的移动动作才能重新开始读/写数据。

连续访问：如果当次IO给出的扇区地址与上次IO结束的扇区地址是连续的，那磁头就能很快的开始这次 IO操作，这样的多个IO操作称为连续访问。

因此尽管相邻的两次IO操作在同一时刻发出，但如果它们的请求的扇区地址相差很大的话也只能称为随机访问，而非连续访问。

磁盘是通过机械运动进行寻址的，随机访问需要过多的定位，所以故障率比较高

MySQL与磁盘交互的基本单位

MySQL作为一款应用软件，可以想象成一种特殊的文件系统。它有着更高的IO场景，为了提高基本的IO效率，**MySQL进行IO的基本单位是16KB**。这个基本数据单元，在MySQL这里叫做page（注意和系统的page区分）

共识

- MySQL中的数据文件，是以page为单位保存在磁盘当中的。
- MySQL 的 CURD 操作，都需要通过**计算**，找到对应的插入位置，或者找到对应要修改或者查询数据。
- 而只要涉及计算，就需要CPU参与，而为了便于CPU参与，一定要能够**先将数据移动到内存中**。
- 所以在特定时间内，数据一定是磁盘中有，内存中也有。后续操作完内存数据之后，以特定的刷新策略，刷新到磁盘。而这时，就涉及到磁盘和内存的数据交互，也就是IO了。而此时IO的基本单位就是Page。

- 为了更好的进行上面的操作，MySQL 服务器在内存中运行的时候，在服务器内部，就申请了被称为 Buffer Pool 的大内存空间，来进行各种缓存。其实就是很大的内存空间，来和磁盘数据进行IO交互。
- 为了更高的效率，一定要**尽可能的减少系统和磁盘IO的次数**

索引的理解

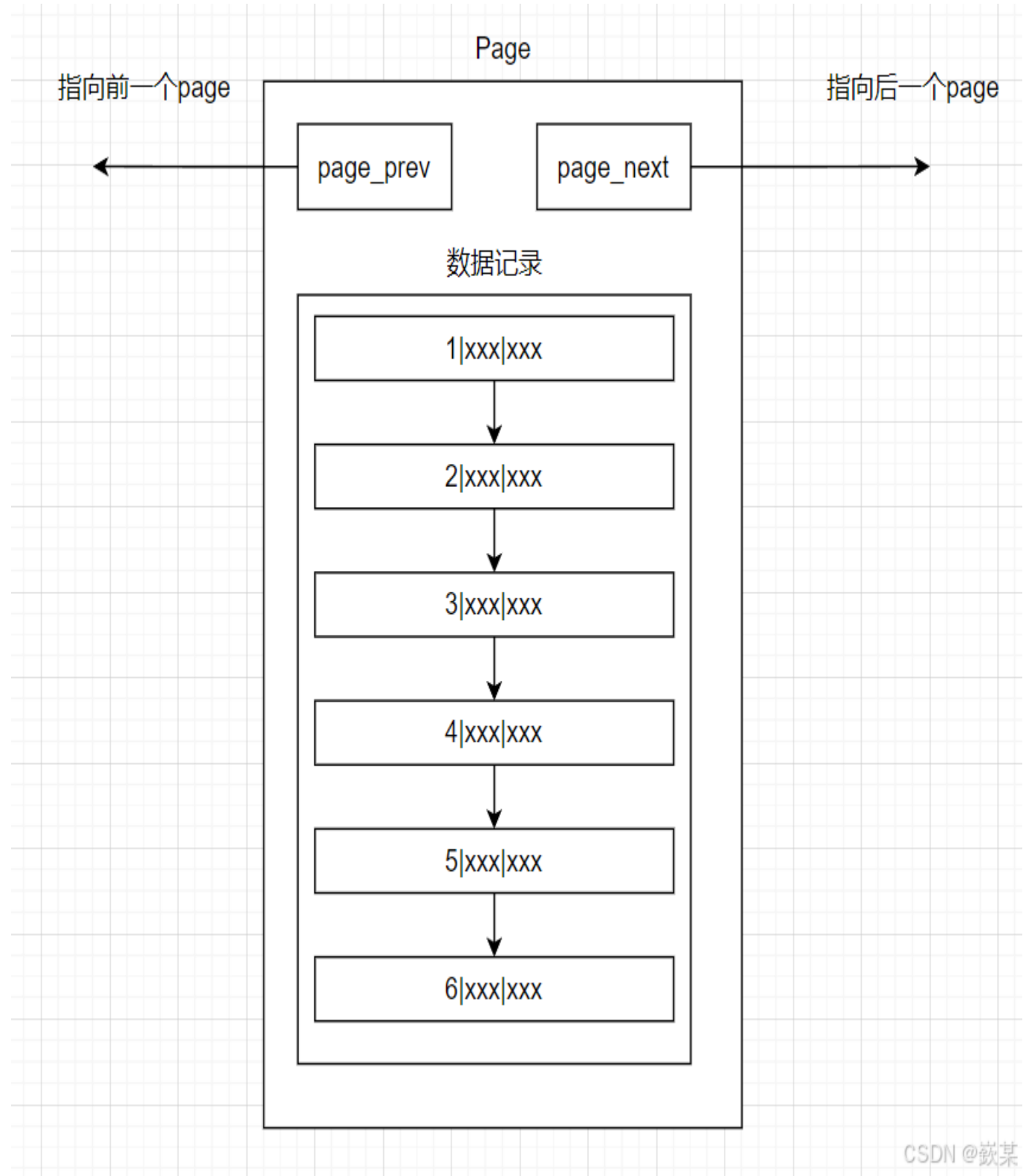
为何MySQL与磁盘IO交互要以page为单位，为什么不用多少加载多少呢？

假如我们在查id为1的记录，查完了马上又要查id为2的记录，用多少加载多少的话，这样就需要两次IO，如果我们把id为1到10都存在一个page里面。一次性加载进来，后面如果再需要查的话就直接在内存里面读取，就不用再进行磁盘IO了。

那你怎么能保证，用户下次要查的数据就在这个page里面呢？我们肯定不能严格保证，但有很大概率，因为计算机的**局部性原理**：即在**一段时间内**，程序的执行通常会集中在**某个较小的内存区域内**。

往往IO效率低下的最主要矛盾不是单词IO的数据量大小，而是IO次数。

理解单个page



CSDN @ 嵌某

在MySQL中，不同的page都是16KB，使用prev和next构成双向链表。

因为有主键，MySQL会默认按照主键给我们的数据进行排序，从上面的Page内数据记录可以看出，**数据是有序并且彼此关联的。**

插入数据时对数据进行排序，是为了**方便查找**，一个page里面存放数据的模块，实质上也是一个链表结构，**链表的特点就是增删快，查询修改慢，所以优化查询效率是必须的。**

因为有序，从前往后查找的每一次都是有效查找，没有任何一次查找是浪费的，并且有可能提前结束查找过程。

理解多个page

- 通过上面的分析，我们知道，上面页模式中，只有一个功能，就是**在查询某条数据的时候直接将一整页的数据加载到内存中，以减少硬盘IO次数，从而提高性能。**但是，我们也可以看到，现在的页模式内部，实际上是采用了链表的结构，前一条数据指向后一条数据，本质上还是通过数据的逐条比较来取出特定的数据。
- 如果有1千万条数据，一定需要多个Page来保存1千万条数据，多个Page彼此使用双链表链接起来，而且每个Page内部的数据也是基于链表的。那么，查找特定一条记录，也一定是线性查找。这效率也太低了。

页目录

我们在看《谭浩强

C

程序设计》这本书的时候，如果我们要看

<

指针章节

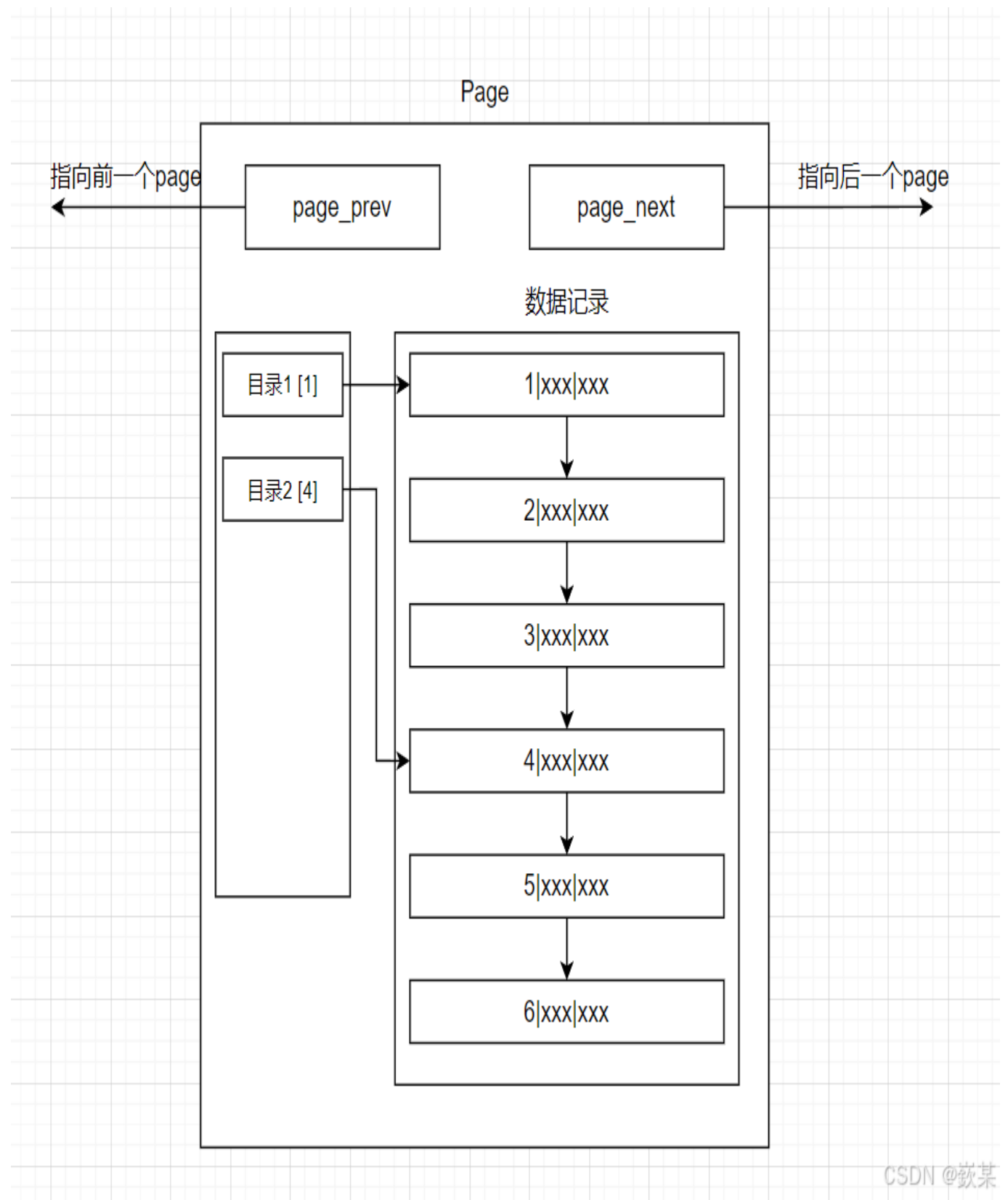
>

，找到该章节有两种做法

- 从头逐页的向后翻，直到找到目标内容
- 通过书提供的目录，发现指针章节在234页(假设)，那么我们便直接翻到234页。同时，查找目录的方案，可以顺序找，不过因为目录肯定少，所以可以快速提高定位
- 本质上，书中的目录，是多花了纸张的，但是却提高了效率
- 所以，**目录，是一种“空间换时间的做法”**

单页情况

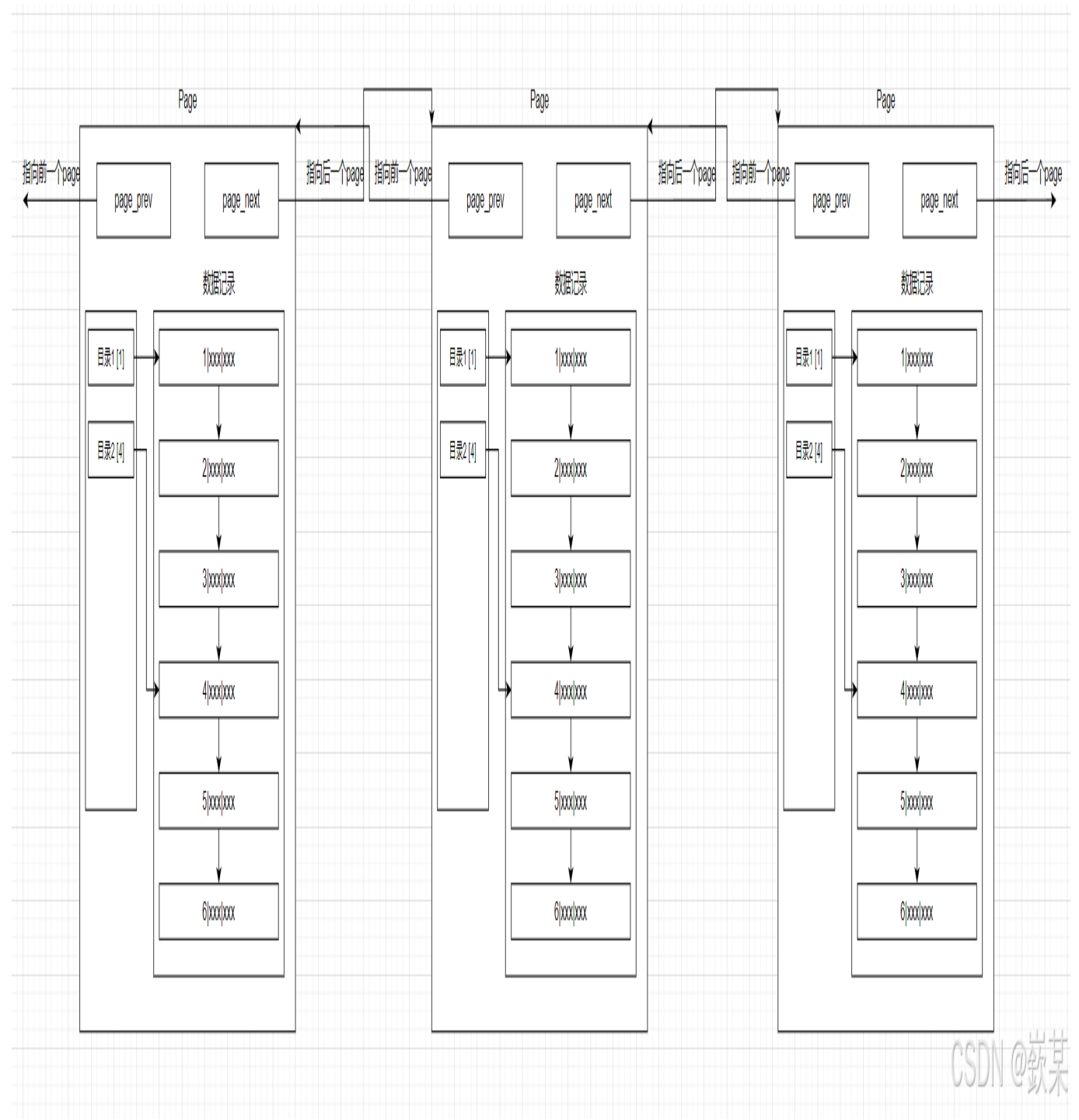
所以针对上面的单页page我们也可以引入目录。



当前，我们在一个page内部引入了目录。如果这是我们要查找id为4的数据，之前必须遍历4次，才能拿到结果，现在直接通过目录2定位新的起始位置，提高了效率。现在可以正式回答为何MySQL会对插入的数据进行键值自动排序？因为可以方便引入目录。

多页情况

MySQL
中每一页的大小只有
16KB
, 单个
Page
大小固定, 所以随着数据量不断增大,
16KB
不可能存下所有的数据, 那么必定会有多个页来存储数据。



在单表数据不断被插入的情况下，MySQL 会在容量不足的时候，自动开辟新的Page来保存新的数据，然后通过指针的方式，将所有的Page组织起来。

需要注意，上面的图，是理想结构，目前要保证整体有序，那么新插入的数据，不一定会在新Page上面，这里仅仅做演示。

这样，我们就可以通过多个

Page

遍历，

Page

内部通过目录来快速定位数据。可是，貌似这样也有效率问题，在Page

之间，也是需要

MySQL

遍历的，遍历意味着依旧需要进行大量的

IO

，将下一个

Page

加载到内存，进行线性检测。这样就显得我们之前的Page

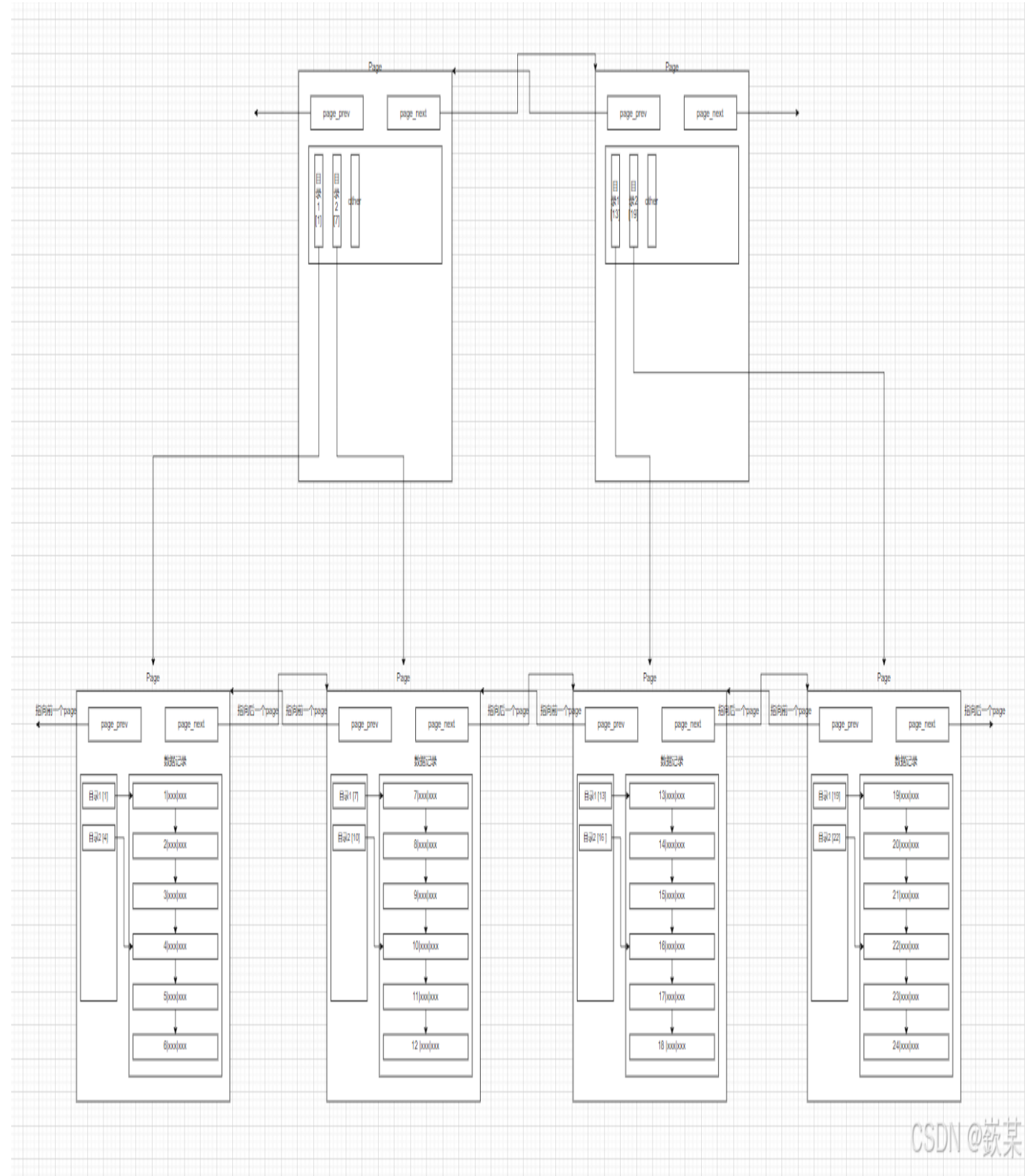
内部的目录，有点杯水车薪了。

那么如何解决呢？解决方案，其实就是我们之前的思路，给

Page

也带上目录。

- 使用一个目录项来指向某一页，而这个目录项存放的就是将要指向的页中存放的最小数据的键值。
- 和页内目录不同的地方在于，这种目录管理的级别是页，而页内目录管理的级别是行。
- 其中，每个目录项的构成是：键值+指针。图中没有画全。

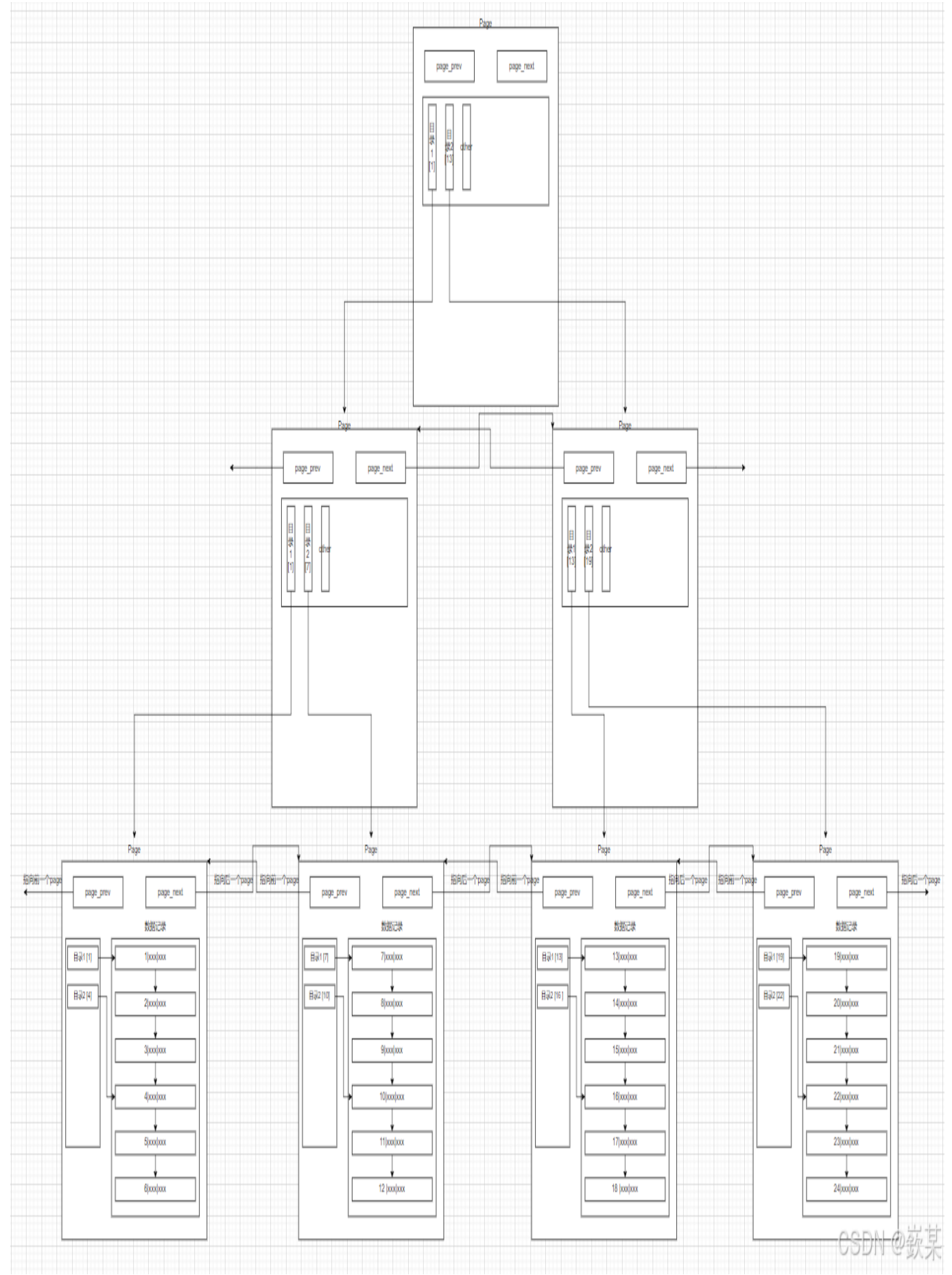


存在一个目录页来管理页目录，目录页中的数据存放的就是指向的那一页中最小的数据。有数据，就可通过比较，找到该访问那个Page，进而通过指针，找到下一个Page。

其实

目录页的本质也是页，普通页中存的数据是用户数据，而目录页中存的数据是普通页的地址。

可是，我们每次检索数据的时候，该从哪里开始呢？虽然顶层的目录页少了，但是还要遍历啊？不用担心，可以在加目录页



这货就是

传说中的B+树啊！

没错，至此，我们已经给我们的表
user
构建完了主键索引。

随便找一个

id=

？我们发现，现在查找的

Page

数一定减少了，也就意味着

IO

次数减少了，那么效率也就提高了。

Page分为目录页和数据页。目录页只放各个下级Page的最小键值。

那么查找的时候，自定向下找，只需要加载部分目录页到内存，即可完成算法的整个查找过程，大大减少了IO次数

InnoDB 在建立索引结构来管理数据的时候，其他数据结构为何不行？

- 链表？线性遍历
- 二叉搜索树？退化问题，可能退化成为线性结构
- AVL &&红黑树？虽然是平衡或者近似平衡，但是毕竟是二叉结构，相比较多阶B+，意味着树**整体过高**，大家都是**自顶向下找，层高越低，意味着系统与硬盘更少的IO Page交互**。虽然你很秀，但是有更秀的。
- Hash？官方的索引实现方式中，MySQL 是支持HASH的，不过 InnoDB 和 MyISAM 并不支持.Hash跟进其算法特征，决定了虽然有时候也很快(O(1))，不过，在面对**范围查找**就明显不行，另外还有其他差别。

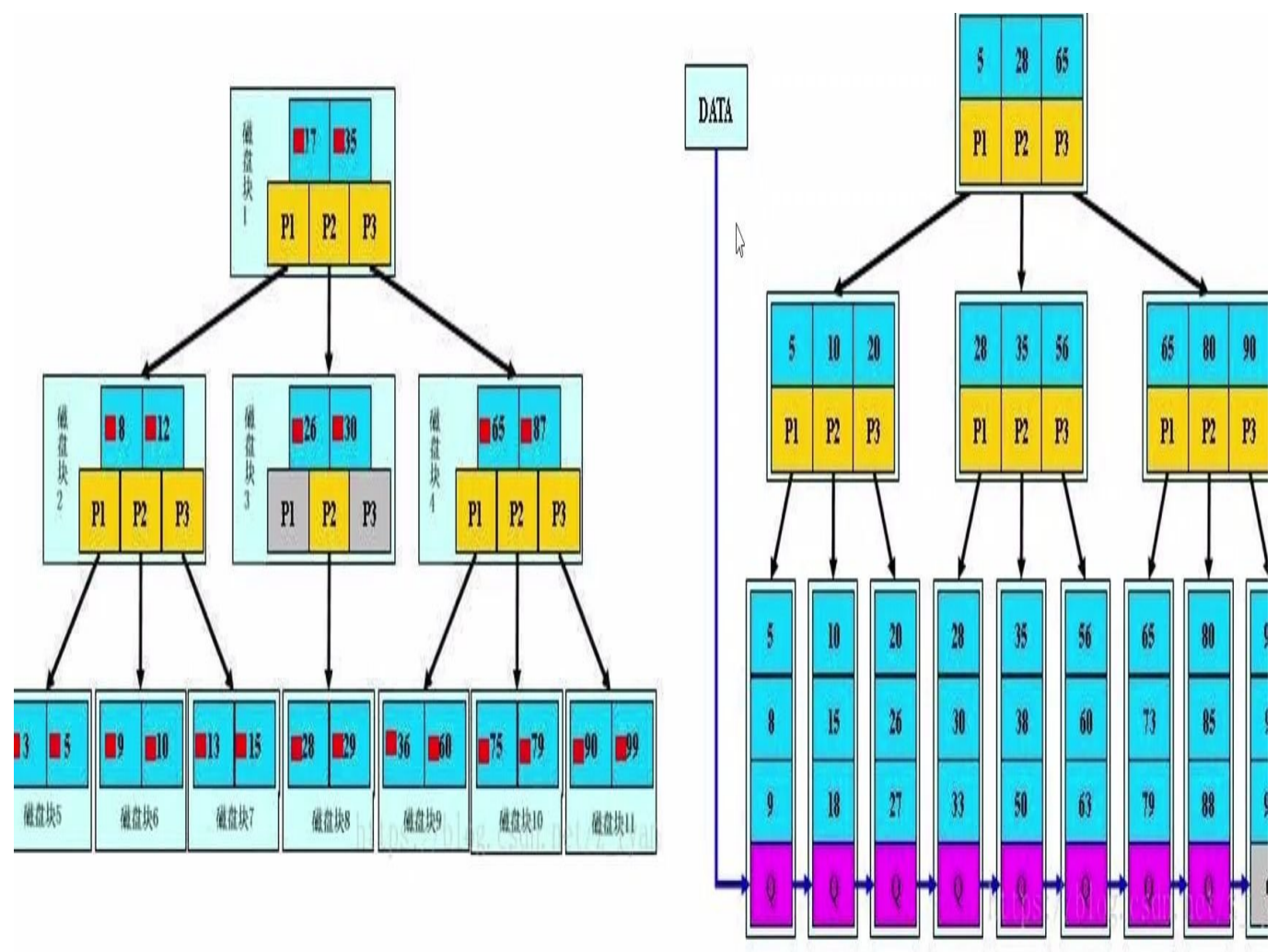
Table 13.1 Index Types Per Storage Engine

Storage Engine	Permissible Index Types
<u>InnoDB</u>	BTREE http://blog.csdn.net/doctor_who2004
<u>MyISAM</u>	BTREE
<u>MEMORY</u> /HEAP	HASH, BTREE
<u>NDB</u>	HASH, BTREE (see note in text)

CSDN @ 嵌某

B vs B+

B树与B+树(图片来源于网络)





激活 Windows
转到设置以激活 Windows。



CSDN @ 某某

这两颗树的主要区别是

- B树节点，既有数据，又有Page指针，而B+，只有叶子节点有数据，其他目录页，只有键值和Page指针
- B树插入和删除操作可能导致节点的分裂和合并，并且可能影响到多个层次的节点，操作相对复杂，维护成本较高。
- B+树非叶子节点不存储数据，相同大小的节点可以容纳更多的键值，从而减少了树的高度。
- B+叶子节点，全部相连，而B没有

为何选择

B+

- 节点不存储data，这样一个节点就可以存储更多的key。可以使得树更矮，所以IO操作次数更少。
- 叶子节点相连，更便于进行范围查找

聚簇索引和非聚簇索引

MyISAM

存储引擎

-

主键索引

MyISAM

引擎同样使用

B+

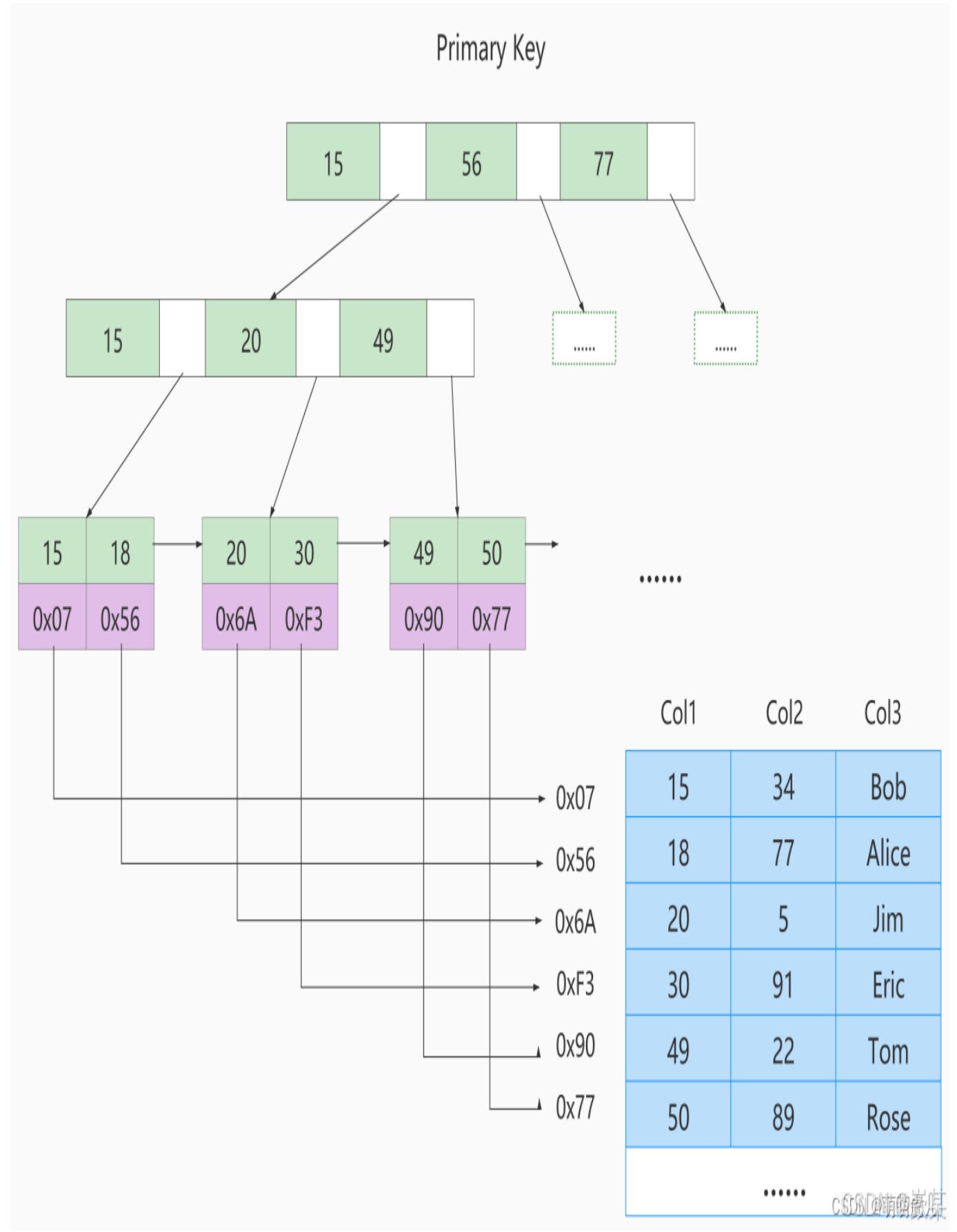
树作为索引结果，

叶节点的data域存放的是数据记录的地址。

下图为

MyISAM 表的主索引， Col1

为主键。



其中， MyISAM 最大的特点是， 将索引Page和数据Page分离， 也就是叶子节点没有数据， 只有对应数据的地址。

相较于 MyISAM索引，InnoDB 是将索引和数据放在一起的。

```
--终端A
mysql> create database myisam_test; --创建数据库
Query OK, 1 row affected (0.00 sec)
mysql> use myisam_test;
Database changed
mysql> create table mtest(
-> id int primary key,
-> name varchar(11) not null
-> )engine=MyISAM; --使用engine=MyISAM
Query OK, 0 rows affected (0.01 sec)
--终端B
[root@VM-0-3-centos mysql]# ls myisam_test/ -al --mysql数据目录下
total 28
drwxr-x--- 2 mysql mysql 4096 Jun 13 13:33 .
drwxr-x--x 13 mysql mysql 4096 Jun 13 13:32 ..
-rw-r----- 1 mysql mysql 61 Jun 13 13:32 db.opt
-rw-r----- 1 mysql mysql 8586 Jun 13 13:33 mtest.frm --表结构数据
-rw-r----- 1 mysql mysql 0 Jun 13 13:33 mtest.MYD --该表对应的数据，当前没有数据，所以是0
-rw-r----- 1 mysql mysql 1024 Jun 13 13:33 mtest.MYI --该表对应的主键索引数据
```

其中，MyISAM 这种用户数据与索引数据分离的索引方案，叫做非聚簇索引

```
--终端A
mysql> create database innodb_test; --创建数据库
Query OK, 1 row affected (0.00 sec)
mysql> use innodb_test;
Database changed
mysql> create table itest(
-> id int primary key,
-> name varchar(11) not null
-> )engine=InnoDB; --使用engine=InnoDB
Query OK, 0 rows affected (0.02 sec)
--终端B
[root@VM-0-3-centos mysql]# ls innodb_test/ -al
total 120
drwxr-x--- 2 mysql mysql 4096 Jun 13 13:39 .
drwxr-x--x 14 mysql mysql 4096 Jun 13 13:38 ..
-rw-r----- 1 mysql mysql 61 Jun 13 13:38 db.opt
```

```
-rw-r----- 1 mysql mysql 8586 Jun 13 13:39 itest.frm --表结构数据
-rw-r----- 1 mysql mysql 98304 Jun 13 13:39 itest.ibd --该表对应的主键索引和用户
数据，虽然现在一行数据没有，但是该表并不为0，因为有主键索引数据
```

其中，InnoDB 这种用户数据与索引数据在一起索引方案，叫做聚簇索引

当然，

MySQL

除了默认会建立主键索引外，我们用户也有可能建立按照其他列信息建立的索引，一般这种索引可以叫做**辅助（普通）索引**。

对于

MyISAM

,

建立辅助（普通）索引和主键索引没有差别，无非就是主键不能重复，而非主键可重复。

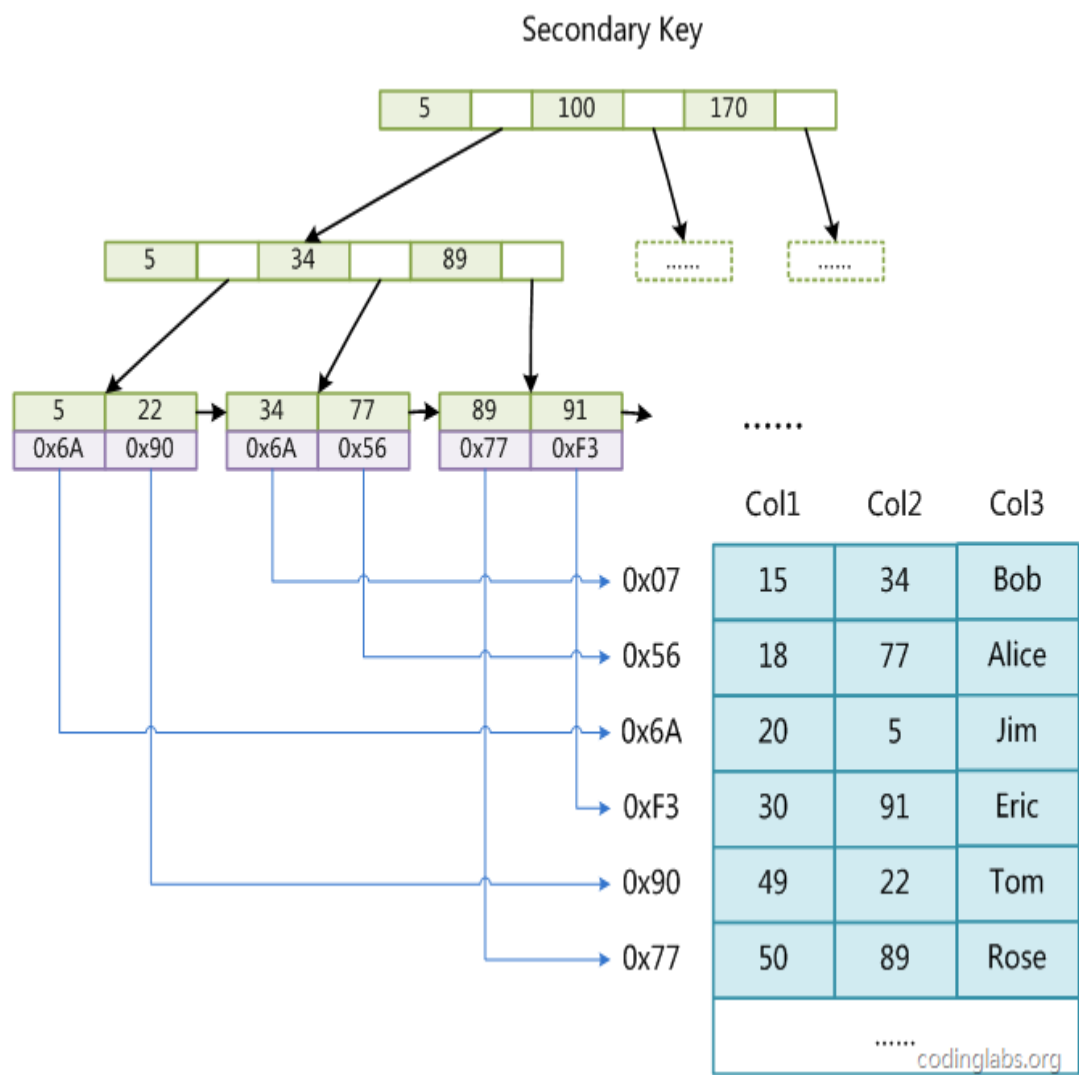
下图就是基于

MyISAM

的

Col2

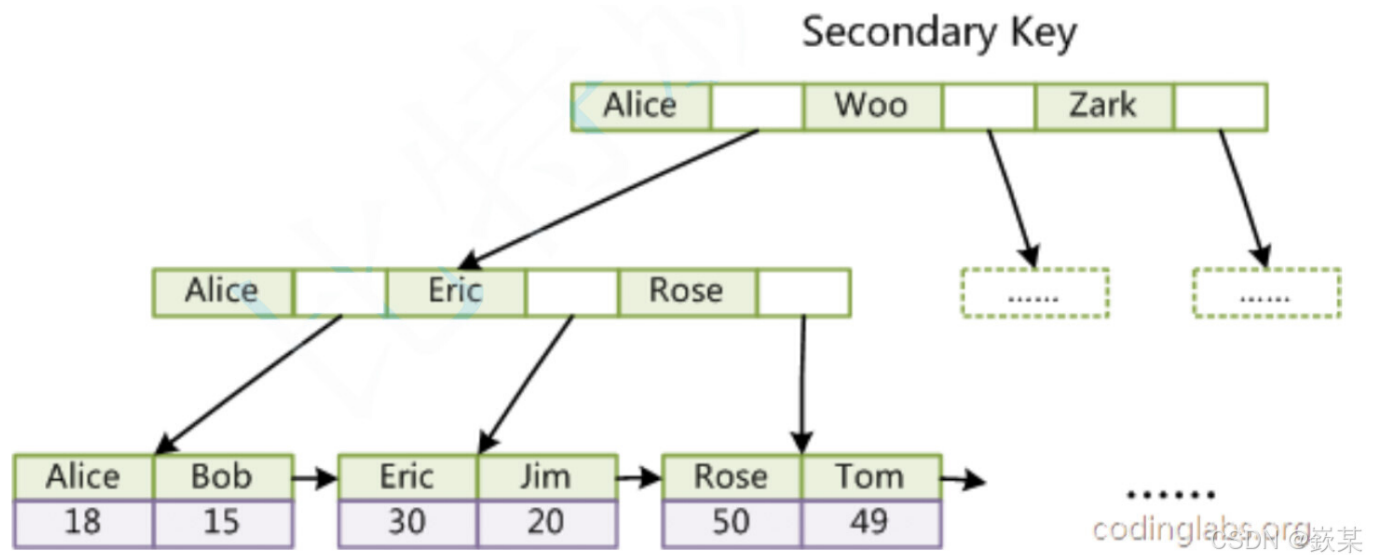
建立的索引，和主键索引没有差别



CSDN 博客

同样， InnoDB 除了主键索引，用户也会建立辅助（普通）索引，我们以上表中的 Col3 建立对应的辅助.

索引如下图：



可以看到，
InnoDB 的非主键索引中叶子节点并没有数据，而只有对应记录的key值。

所以通过辅助（普通）索引，找到目标记录，需要两遍索引：首先检索辅助索引获得主键，然后用主键到主索引中检索获得记录。这种过程，就叫做
回表查询

为何
InnoDB
针对这种辅助（普通）索引的场景，不给叶子节点也附上数据呢？原因就是太浪费空间了。

索引操作

创建主键索引

方式一

```
-- 在创建表的时候，直接在字段名后指定 primary key
create table user1(id int primary key, name varchar(30));
```

方式二

```
-- 在创建表的最后，指定某列或某几列为主键索引
create table user2(id int, name varchar(30), primary key(id));
```

方式三

```
create table user3(id int, name varchar(30));
-- 创建表以后再添加主键
alter table user3 add primary key(id);
```

主键索引的特点

- 一个表中，最多有一个主键索引，当然可以是复合主键
- 主键索引的效率高（主键不可重复）
- 创建主键索引的列，它的值不能为null，且不能重复
- 主键索引的列基本上是int

唯一索引的创建

方式一

```
-- 在表定义时，在某列后直接指定unique唯一属性。
create table user4(id int primary key, name varchar(30) unique);
```

方式二

```
-- 创建表时，在表的后面指定某列或某几列为unique  
create table user5(id int primary key, name varchar(30), unique(name));
```

方式三

```
create table user6(id int primary key, name varchar(30)) ;  
alter table user6 add unique(name);
```

唯一索引的特点

- 一个表中，可以有多个唯一索引
- 查询效率高
- 如果在某一列建立唯一索引，必须保证这列不能有重复数据
- 如果一个唯一索引上指定not null，等价于主键索引

普通索引的创建

方式一

```
create table user8(id int primary key,  
    name varchar(20),  
    email varchar(30),  
    index(name) --在表的定义最后，指定某列为索引  
);
```

方式二

```
create table user9(id int primary key, name varchar(20), email
varchar(30));
alter table user9 add index(name); --创建完表以后指定某列为普通索引
```

方式三

```
create table user10(id int primary key, name varchar(20), email
varchar(30));
-- 创建一个索引名为 idx_name 的索引
create index idx_name on user10(name);
```

普通索引的特点

- 一个表中可以有多个普通索引，**普通索引在实际开发中用的比较多**
- 如果某列需要创建索引，但是该列有重复的值，那么我们就应该使用普通索引

全文索引的创建

当对文章字段或有大量文字的字段进行检索时，会使用到全文索引。MySQL提供全文索引机制，但是有要求，要求表的存储引擎必须是MyISAM，而且默认的全文索引支持英文，不支持中文。如果对中文进行全文检索，可以使用sphinx的中文版(coreseek)。

```
CREATE TABLE articles (
  id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,
  title VARCHAR(200),
  body TEXT,
  FULLTEXT (title,body)
)engine=MyISAM;
```

```
INSERT INTO articles (title,body) VALUES
('MySQL Tutorial','DBMS stands for DataBase ...'),
('How To Use MySQL Well','After you went through a ...'),
('Optimizing MySQL','In this tutorial we will show ...'),
('1001 MySQL Tricks','1. Never run mysqld as root. 2. ...'),
('MySQL vs. YourSQL','In the following database comparison ...'),
('MySQL Security','When configured properly, MySQL ...');
```

- 查询有database的数据

```
mysql> select * from articles where body like '%database%';
```

如果使用这种查询方式，虽然查询出数据，但是没有使用到全文索引

可以用explain工具看一下，是否使用到索引

```
mysql> explain select * from articles where body like '%database%'\G
***** 1. row *****
id: 1
select_type: SIMPLE
table: articles
type: ALL
possible_keys: NULL
key: NULL          <== key为null表示没有用到索引
key_len: NULL
ref: NULL
rows: 6
Extra: Using where
1 row in set (0.00 sec)
```

如何使用全文索引呢？

```
mysql> SELECT * FROM articles
      -> WHERE MATCH (title,body) AGAINST ('database');
+----+-----+-----+-----+
| id | title                | body                                     |
+----+-----+-----+-----+
| 5  | MySQL vs. YourSQL    | In the following database comparison ... |
| 1  | MySQL Tutorial       | DBMS stands for DataBase ...           |
+----+-----+-----+-----+
```

通过explain来分析这个sql语句

```
mysql> explain SELECT * FROM articles WHERE MATCH (title,body) AGAINST
('database')\G
***** 1. row *****
id: 1
select_type: SIMPLE
table: articles
type: fulltext
possible_keys: title
key: title          <= key用到了title
key_len: 0
ref:
rows: 1
Extra: Using where
```

查询索引

方式一

```
show keys from 表名
```

```
mysql> show keys from goods\G
***** 1. row *****
```

```
Table: goods <= 表名
Non_unique: 0 <= 0表示唯一索引
Key_name: PRIMARY <= 主键索引
Seq_in_index: 1
Column_name: goods_id <= 索引在哪列
Collation: A
Cardinality: 0
Sub_part: NULL
Packed: NULL
Null:
Index_type: BTREE <= 以b树（其实就是b+树）形式的索引
Comment:
1 row in set (0.00 sec)
```

方式二

```
show index from 表名;
```

方式三

```
desc 表名;
```

删除索引

方式一 -- 删除主键索引

```
alter table 表名 drop primary key;
```

方式二 -- 删除其他索引

```
alter table 表名 drop index  
索引名; 索引名就是show keys from 表名中的 Key_name 字段
```

方式三

```
drop index 索引名 on 表名
```

索引创建原则

- 比较频繁作为查询条件的字段应该创建索引
- 唯一性太差的字段不适合单独创建索引，即使频繁作为查询条件
- 更新非常频繁的字段不适合创建索引
- 不会出现在where子句中的字段不该创建索引

复合索引

复合索引是一种数据库索引类型，它由表中的多个列组合而成，用于提高在这些列上进行查询的性能。以下从多个方面详细介绍复合索引：

定义与原理

在数据库中，当单个列的索引无法满足复杂查询的性能需求时，就可以创建复合索引。复合索引将多个列按照特定的顺序组合在一起，形成一个索引结构。数据库系统在使用复合索引时，会根据索引中列的顺序依次进行匹配和查找，从而快速定位到满足条件的行。

示例

假设有一个“员工信息表”（employees），包含“姓名”（name）、“部门”（department）、“入职时间”（hire_date）等列。如果经常需要执行类似“查询某个部门中入职时间在特定范围内的员工”这样的查询：

```
SELECT * FROM employees
WHERE department = '销售部' AND hire_date BETWEEN '2020-01-01' AND '2023-12-31';
```

为了提高这个查询的效率，可以在“department”和“hire_date”列上创建复合索引：

```
CREATE INDEX idx_department_hire_date ON employees (department, hire_date);
```

在这个复合索引中，“department”是第一列，“hire_date”是第二列。数据库在执行查询时，会先根据“department”列进行筛选，缩小查询范围，然后再在筛选后的结果中根据“hire_date”列进一步筛选，从而快速找到符合条件的员工记录。

注意事项

- **索引列顺序**：复合索引中列的顺序非常重要。一般来说，将选择性高（即该列的值具有较多不同的取值）的列放在前面，这样可以更有效地缩小查询范围。例如，如果“department”列的取值种类比“hire_date”列多，那么将“department”列放在复合索引的第一列。
- **最左前缀原则**：复合索引遵循**最左前缀原则**，即只有查询条件中使用了复合索引的最左边的列时，索引才会被使用。例如，上述创建的复合索引“idx_department_hire_date”，如果查询语句是“SELECT * FROM employees WHERE hire_date BETWEEN '2020-01-01' AND '2023-12-31';”，由于没有使用到“department”列（复合索引的最左边列），这个复合索引将不会被使用。
- **维护成本**：复合索引虽然可以提高查询性能，但也会增加索引的维护成本。每次对表中的数据进行插入、更新或删除操作时，数据库不仅要更新表数据，还要更新相应的复合索引，因此需要谨慎使用，避免创建过多不必要的复合索引。
- **索引覆盖**：如果查询列都包含在复合索引里，数据库引擎可以直接从该索引中获取满足条件的数据，不会从其他索引中获取。但这会增加维护成本和存储空间。

本期博客到这里就结束了，如果有什么错误，欢迎指出，如果对你有帮助，请点个赞，谢谢！